



Operating Systems

Assignment # 3

Instructor	Dr. Rana Asif Rehman
Section	BCS 4A + 4B
Teaching Assistant	Muhammad Ali Butt
Semester	Spring 2025

Fast School of Computing

FAST-NU, Lahore, Pakistan

Instructions:

- Follow the assignment solution submission guidelines specified on Google Classroom (AssignmentSolution.zip).
- Comment your code to keep it readable.
- Plagiarism is not acceptable.
- For your ease, we will use threads to create shared memory scenarios. Regardless, you should note that multiple processes may also utilize shared memory.

Question No. 1

In this question, we warm up our process synchronization knowledge and syntax.

The “Red Light, Green Light” game we programmed in Assignment 2 had a race condition problem. Modify its code to enforce mutual exclusion/prevent race conditions.

Question No. 2

In this question, we shall explore a classic synchronization challenge.

Suppose that in a national park, a canyon is spanned by a singular rope (with the ends labelled as Point A and Point B). Chimpanzees use this rope to cross the canyon by swinging hand-over-hand. This scenario presents two critical constraints:

1. If chimpanzees traveling in opposite directions meet midway, they cannot pass each other safely on the narrow rope.
2. The rope can only safely support a maximum of 5 chimpanzees at once before snapping.

Thankfully, the intelligent chimpanzees (represented as threads) use semaphores for safe and fair coordination. The goal is to design a synchronization scheme using semaphores that ensures:

1. Ordered Crossing: Chimpanzees arriving at point A line up and cross the rope to point B in the order they arrive. Same goes for chimpanzees going from Point B to Point A.
2. Rope load: No more than five chimpanzees on the rope at any given time.
3. Crossing Guarantee: Once a chimpanzee process begins to cross the rope, it is guaranteed to get to the other side without running into another chimpanzee going the other way.
4. Streaming Guarantee: If multiple chimpanzees arrive to point A, and, there are no chimpanzees at point B, they can all cross in sequence, subject to the rope load and fairness restrictions. Same goes for chimpanzees going from Point B to Point A.
5. Fairness (no Starvation): A continuing stream of chimpanzees crossing in one direction should not bar chimpanzees from crossing the other way indefinitely. So, after every 10 chimpanzees complete their crossing from A to B (or vice versa), if chimpanzees are

waiting to cross from B to A, the crossing direction must switch into a B-to-A crossing (or vice versa).

Create a program that takes a command line argument *n*, representing the number of chimpanzees. Use **pthread_detach(...)** to create each thread (chimpanzee).

Lastly, each chimpanzee should announce (print) its arrival. Upon its turn, each chimpanzee should announce that it is about to cross the canyon, sleep for 0.5 seconds, and then announce that it has crossed the canyon. Print threadID with each message too.

Question No. 3

In this question, we have to explore or invent a new synchronization algorithm.

The “**Peterson’s Solution**” is an impressive synchronization algorithm, ensuring **Mutual Exclusion**, **Progress** as well as **Bounded Waiting**, but it only works for 2 processes.

In this question, you have to implement an algorithm that synchronizes more than 2 processes. Your solution will be evaluated based on how many of the three essential conditions (Mutual Exclusion, Progress, and Bounded Waiting) it satisfies.

Note: You are not allowed to use the Bakery Algorithm or Semaphores.

Deliverables:

1. An implementation of your algorithm. The critical section of the program should be a simple integer variable. The program should input 2 command line arguments, ‘plus’ and ‘minus’, indicating the number of threads that will attempt to increment and decrement the variable, respectively.
2. An explanation of how your algorithm works (in a separate Word file). You may add images and examples. Make it neat.

Best of luck!